1.5 Regularized linear models

- we just saw that maximum likelihood estimation is prone to overfitting
- we often observe that the magnitude of the parameter values becomes *relatively large* if we run into overfitting
- to mitigate the effect of huge parameter values, we can place a *prior distribution* $p(\boldsymbol{w})$ on the parameters
- the prior distribution explicitly encodes what parameter values are plausible (before having seen any data)
- for example, a Gaussian prior $w \sim \mathcal{N}(0, 1)$ on a single parameter encodes that parameter values are expected to lie in the interval $[-2, 2]$ (two standard deviations around the mean value)

1.5 Regularized linear models

- once a dataset $\boldsymbol{X}, \boldsymbol{y}$ is available, instead of maximizing the likelihood, we seek parameters that maximize the posterior distribution $p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y})$
- this procedure is called *maximum a posteriori* estimation
- the posterior over the weights $\boldsymbol{w}$, given the training data $\boldsymbol{X}, \boldsymbol{y}$, is obtained by applying Bayes' theorem as:

$$p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) = \frac{p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w})p(\boldsymbol{w})}{p(\boldsymbol{y}|\boldsymbol{X})}.$$

- to find the maximum a posteriori estimation, we minimize the *negative log-posterior distribution* with respect to $\boldsymbol{w}$:

$$-\log p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) = -\log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) - \log p(\boldsymbol{w}) + \log p(\boldsymbol{y}|\boldsymbol{X}). \tag{10}$$

- we see that the log-posterior in (10) is the sum of the log-likelihood $\log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w})$ and the log-prior $\log p(\boldsymbol{w})$, so that the maximum a posteriori estimate will be a "compromise" between the prior (our suggestion for plausible parameter values before observing data) and the data-dependent likelihood

1.5 Regularized linear models

- first, assume that the weights are normally distributed with variance τ, $\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \tau^2 \boldsymbol{I})$, which implies that:

$$-\log p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y}) = \frac{1}{2\sigma^2}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^\top (\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + \frac{1}{2\tau^2}\boldsymbol{w}^\top \boldsymbol{w} + \text{const.} \tag{11}$$

  where const. represent terms that are independent of $\boldsymbol{w}$
- here, the first term corresponds to the contribution from the log-likelihood, and the second term originates from the log-prior
- by taking the gradient of (11) and making it equal with $\boldsymbol{0}^\top$, the maximum a posteriori estimate for $\boldsymbol{w}$ will be:

$$\boldsymbol{w}^* = (\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \boldsymbol{X}^\top \boldsymbol{y},$$

  where we denoted $\lambda := \frac{\sigma^2}{\tau^2}$

1.5 Regularized linear models

- similarly as for maximum likelihood estimation, we can define a loss corresponding to maximum a posteriori estimation:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + \frac{1}{2}\lambda||\boldsymbol{w}||_2^2,$$

  where $||\boldsymbol{w}||_2 = \left(\sum_{j=0}^{d} |w_j|^2\right)^{1/2}$ is the ℓ_2 norm of $\boldsymbol{w}$
- linear regression using this loss function is called *ridge regression*
- ridge regression is a *regularized* version of linear regression: a regularization term equal to $\frac{1}{2}\lambda||\boldsymbol{w}||_2^2$ is added to the loss function
- this forces the learning algorithm to not only fit the data but also keep the model weights as *small* as possible
- note that the regularization term should only be added to the loss function during training
- once the model is trained, we want to use the unregularized performance measure to evaluate the model's performance

1.5 Regularized linear models

- the hyperparameter λ controls how much we want to regularize the model
- if $\lambda = 0$, then ridge regression is just linear regression
- if λ is very large, then all weights end up very close to zero and the result is a flat line going through the data's mean
- as with linear regression, we can perform ridge regression either by computing a closed-form equation or by performing gradient descent
- the pros and cons are the same; for gradient descent, we just add $\lambda \boldsymbol{w}$ to the gradient vector:

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \frac{1}{n} \boldsymbol{X}^{\top} (\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + \lambda \boldsymbol{w}.$$

- it is important to *scale* the data before performing ridge regression, as it is sensitive to the scale of the input features, which is true of most regularized models

1.5 Regularized linear models

- on the other hand, we can assume a Laplace distribution for the weights, $\boldsymbol{w} \sim \text{Lap}(\boldsymbol{0}, b\boldsymbol{I})$
- the probability density for the Laplace distribution is given as:

$$p(x|\mu, b) = \frac{1}{2b} \exp\left(-\frac{1}{b}|x - \mu|\right).$$

- here, μ is a location parameter and $b > 0$ is a scale parameter
- the loss corresponding to the Laplace prior on the weights is:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + \lambda||\boldsymbol{w}||_1,$$

  where $||\boldsymbol{w}||_1 = \sum_{j=0}^{d} |w_j|$ is the ℓ_1 norm of $\boldsymbol{w}$
- linear regression using this loss function is called *lasso regression*

1.5 Regularized linear models

- *Least Absolute Shrinkage and Selection Operator Regression* (usually simply called lasso regression) is another regularized version of linear regression: just like ridge regression, it adds a regularization term to the loss function, but it uses the ℓ_1 norm of the weight vector instead of half the square of the ℓ_2 norm
- an important characteristic of lasso regression is that it tends to eliminate the weights of the least important features (i.e., set them to zero)
- in other words, lasso regression automatically performs *feature selection* and outputs a *sparse model*
- that is, it often sets many weights to zero, effectively declaring the corresponding attributes to be completely irrelevant
- models that discard attributes can be easier for a human to understand, and may be less likely to overfit

1.5 Regularized linear models

- Figure 16 gives an intuitive explanation of why ℓ_1 regularization leads to weights of zero, while ℓ_2 regularization does not
- note that minimizing $\frac{1}{2n}(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})+\frac{1}{2}\lambda||\boldsymbol{w}||_2^2$ or $\frac{1}{2n}(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})+\lambda||\boldsymbol{w}||_1$ is equivalent to minimizing $\frac{1}{2n}(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})$ subject to the constraint that $\frac{1}{2}\lambda||\boldsymbol{w}||_2^2 \leq c$ or $\lambda||\boldsymbol{w}||_1 \leq c$, for some constant c that is related to λ
- now, in Figure 16(a) the diamond-shaped box represents the set of points $\boldsymbol{w}$ in two-dimensional weight space that have ℓ_1 norm less than c; our solution will have to be somewhere inside this box
- the concentric ovals represent contours of the mean squared error function, with the minimum loss at the center

1.5 Regularized linear models

- we want to find the point in the box that is *closest* to the minimum; we can see from the diagram that, for an arbitrary position of the minimum and its contours, it will be common for the corner of the box to find its way closest to the minimum, just because the corners are pointy
- and of course the corners are the points that have a value of zero in some dimension
- in Figure 16(b), we have done the same for the ℓ_2 norm, which represents a circle rather than a diamond
- here we can see that, in general, there is no reason for the intersection to appear on one of the axes; thus ℓ_2 regularization does not tend to produce zero weights

1.5 Regularized linear models

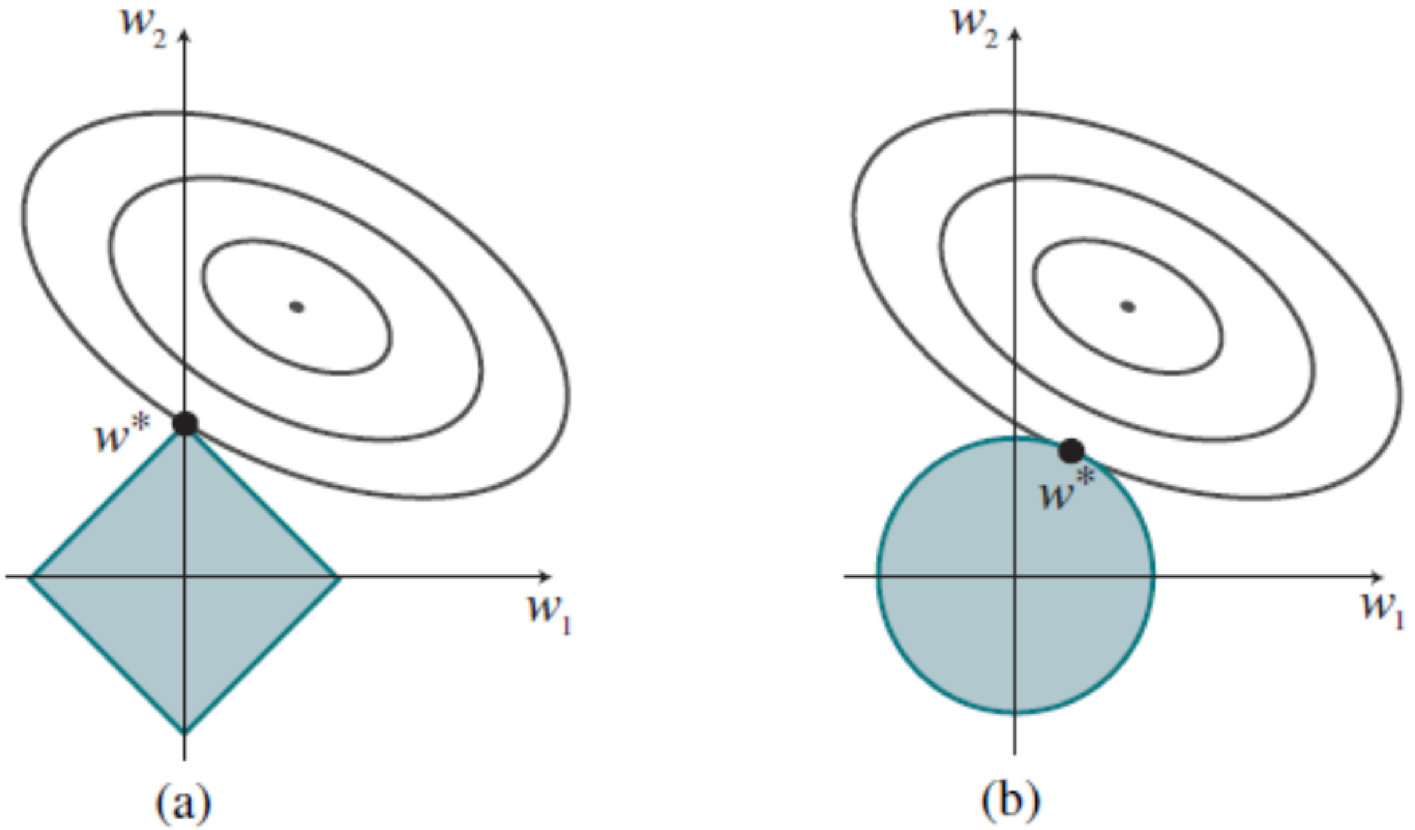


Figure 16: Why ℓ_1 regularization tends to produce a sparse model. Left: with ℓ_1 regularization (box), the minimal achievable loss (concentric contours) often occurs on an axis, meaning a weight of zero. Right: with ℓ_2 regularization (circle), the minimal loss is likely to occur anywhere on the circle, giving no preference to zero weights.

1.5 Regularized linear models

- the lasso loss function is not differentiable whenever $w_j = 0$ for some $j = 0, 1, \ldots, d$, but gradient descent still works fine if we use a *subgradient vector* $\boldsymbol{g}$ instead, when any $w_j = 0$
- we can think of a subgradient vector at a nondifferentiable point as an intermediate vector between the gradient vectors around that point
- a subgradient vector we can use for gradient descent with the lasso loss function is:

$$\boldsymbol{g}(\boldsymbol{w}) = \frac{1}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + \lambda \text{sign}(\boldsymbol{w}),$$

where $$\text{sign}(x) = \begin{cases} -1, & \text{if } x < 0 \\ 0, & \text{if } x = 0 \\ +1, & \text{if } x > 0 \end{cases}$$

1.5 Regularized linear models

- *elastic net* is a middle ground between ridge regression and lasso regression
- the regularization term is a simple mix of both ridge and lasso's regularization terms, and we can control the mix ratio $r \in [0, 1]$
- when $r = 0$, elastic net is equivalent to ridge regression, and when $r = 1$, it is equivalent to lasso regression:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^{\top}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) + r\lambda||\boldsymbol{w}||_1 + \frac{1-r}{2}\lambda||\boldsymbol{w}||_2^2.$$

1.5 Regularized linear models

- so when should we use plain linear regression (i.e., without any regularization), ridge, lasso, or elastic net?
- it is almost always preferable to have at least a *little bit of regularization*, so generally we should avoid plain linear regression
- ridge is a good default, but if we suspect that only a few features are useful, we should prefer lasso or elastic net because they tend to reduce the useless features' weights down to zero, as we have discussed
- in general, elastic net is preferred over lasso, because lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated

1.5 Regularized linear models

- a very different way to *regularize* iterative learning algorithms such as gradient descent is to stop training as soon as the validation error reaches a minimum; this is called *early stopping*
- Figure 17 shows a complex model (in this case, a high-degree polynomial regression model) being trained with batch gradient descent
- as the epochs go by, the algorithm learns, and its prediction error (RMSE) on the training set goes down, along with its prediction error on the validation set
- after a while though, the validation error stops decreasing and starts to go back up
- this indicates that the model has started to overfit the training data

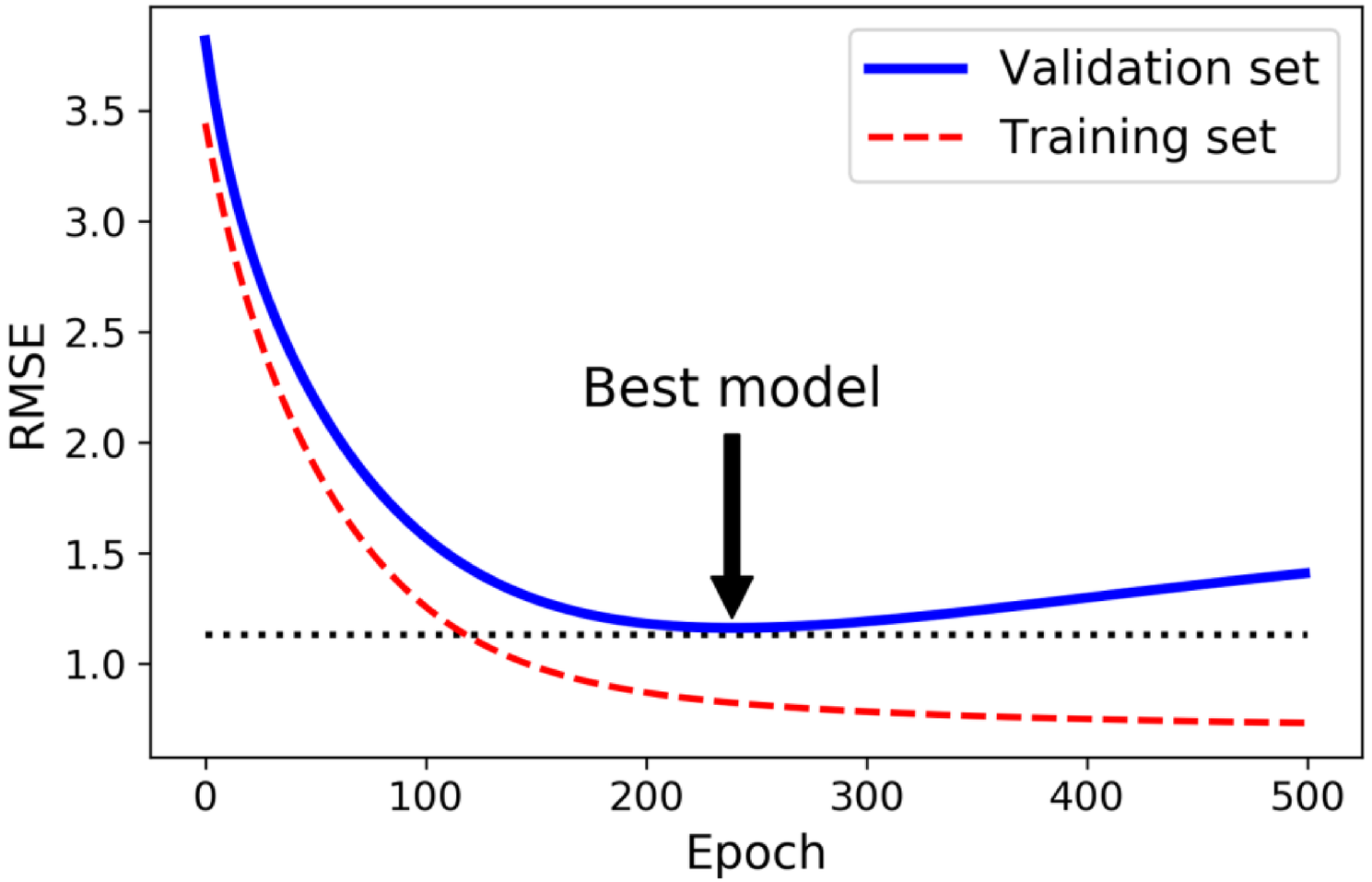


Figure 17: Early stopping regularization.

1.5 Regularized linear models

- with early stopping, we just stop training as soon as the validation error reaches the minimum
- with stochastic and mini-batch gradient descent, the curves are not so smooth, and it may be hard to know whether we have reached the minimum or not
- one solution is to stop only after the validation error has been above the minimum for some time (when we are confident that the model will not do any better), then roll back the model parameters to the point where the validation error was at a minimum